ChatGPT

Frontend UI Implementation – OCR Snipping & Summarization Application

Introduction

This repository contains the **front-end user interface** (UI) for the *OCR Snipping & Summarization* application. The application assists medical associates in uploading scanned PDF documents, selecting regions of interest, extracting text via Optical Character Recognition (OCR) and requesting concise summaries of the extracted text. The UI is built with **React 18** and focuses exclusively on client-side functionality while backend endpoints for cropping, OCR and summarisation are stubbed. By following the guidelines in the Business Requirements Document (BRD), technical report and implementation document, this front-end delivers a responsive and accessible workflow for end users.

Requirements overview

The BRD describes a linear workflow: upload a scanned PDF, snip regions of interest, edit extracted text and request a summary. The UI therefore provides the following capabilities:

- **File upload and validation:** Users can drag-and-drop or use a file input to load a PDF document. The *PDFUploader* component validates the MIME type (`application/pdf`) and enforces a 10 MB size limit; invalid files produce clear error messages. Only valid files are passed to the viewer.
- **PDF display with navigation:** A multi-page viewer renders the uploaded PDF. Thumbnails, zoom controls and page navigation allow users to explore the document. Panning and zooming are supported via mouse or keyboard, and page scale information is exposed for coordinate conversions.
- **Snipping overlay:** Users draw rectangular selections (snips) on any page. The *SnipOverlay* overlay captures mouse and keyboard events, draws a semi-transparent rectangle with resize handles and computes *normalised* coordinates `(x_norm, y_norm, w_norm, h_norm)` by dividing the rectangle's pixel coordinates by the displayed page size. Because PDF coordinate systems originate at the bottom-left, the overlay inverts the y-axis before sending coordinates to the backend.
- **Editable text boxes:** After a snip is finalised, the client calls the stubbed `/api/snip-crop` endpoint. It receives a dummy OCR string and stores each snip as `{id, page, rect, text}`. The *SnipList* component displays an ordered list of text boxes; users can insert new boxes, delete or merge existing boxes and use keyboard shortcuts such as `Ctrl + N` (new box), `Delete` (remove), `Shift + M` (merge) and `Ctrl + Z/Y` (undo/redo). Focus management follows WAI-ARIA Authoring Practices: the container uses `role="list"` and each text box uses `role="listitem"` with proper `tabindex` attributes.
- **Summarisation:** A *SummarizePanel* provides a primary **Summarize** button. It gathers the text from all or selected boxes and sends a POST request to `/api/summarize`, including parameters such as `max_sentences`. While waiting, the button is disabled and shows a spinner; users can also trigger summarisation using `Ctrl + Enter`. Once a response is received, the summary is displayed with options to copy or export it.

- **Accessibility:** Every interactive element has an ARIA role and keyboard equivalent. Colour contrast meets WCAG 2.1 AA standards and a high-contrast mode is provided. The drag-and-drop area is keyboard-operable and dynamic status messages are announced via `aria-live` regions.
- **Security & Validation:** Client-side checks prevent malicious uploads by validating file type and size, sanitising user input and including CSRF tokens for state-changing requests. The UI assumes HTTPS and uses `SameSite=Strict` cookies to protect against cross-site request forgery.

Technology stack

Component	Purpose	Rationale
React 18	Front-end framework	React's functional components and hooks provide fine-grained state management and a rich ecosystem. The Context API suffices for local state; Redux Toolkit can be adopted for future caching needs.
React PDF Viewer	PDF rendering	Built on PDF.js but offers a complete UI with drag-and-drop upload, thumbnails, zoom controls and keyboard navigation. The technical report recommends it over alternatives because it emphasises accessibility and exposes page scale information.
Material UI (MUI)	UI library	Provides accessible form controls, modals, buttons and typography with built-in ARIA attributes and high-contrast themes. Styling uses MUI's `sx` prop and custom modules to ensure scoped styles.
Fetch API	Network calls	API calls are made using `fetch` wrapped in asynchronous functions; stubbed endpoints return hard-coded responses during development.
Mock Service Worker (MSW)	API stubbing	Intercepts calls to `/api/snip-crop` and `/api/summarize` and returns fixed JSON data to simulate backend behaviour. It registers handlers in `src/mocks/browser` and starts the service worker in `src/index.js`.

Application architecture

The UI follows a **modular component structure**, separating concerns into upload/rendering, snipping, text editing and summarisation. Below is an overview of the hierarchy and data flow.

Component hierarchy

Component	Responsibility	Notes
App	Root component; sets up routing and global context.	Provides theme and high-contrast toggle and renders the main layout.

Component	Responsibility	Notes
PDFUploader	Handles drag-and-drop or file selection and performs client-side validation.	Displays error messages when a file is unsupported or exceeds 10 MB.
PDFViewerPane	Wraps `React PDF Viewer` to render pages, thumbnails and zoom controls.	Receives the uploaded file via props and exposes page dimensions via callbacks.
SnipOverlay	Transparent overlay on top of each PDF page.	Captures mouse/keyboard events for creating, resizing and moving rectangles; converts pixel coordinates to normalised coordinates and inverts the y-axis. Supports keyboard resizing and moving; resize handles appear when focused and ARIA roles are assigned.
SnipList	Maintains an ordered list of text boxes corresponding to each snip.	Provides controls to insert, delete or merge boxes and supports keyboard navigation.
SummarizePanel	Contains the summarisation button and displays the returned summary or loading indicator.	Allows copying the summary to the clipboard and exporting it.

Data flow

1. **File upload:** The user drops or selects a PDF; *PDFUploader* validates type and size and passes the file to *PDFViewerPane*.
2. **Rendering & coordinate extraction:** *PDFViewerPane* renders the pages using `React PDF Viewer`. When the user draws a rectangle on *SnipOverlay*, the overlay computes normalised coordinates by dividing pixel coordinates by the displayed page's width/height and inverting the y-axis.
3. **Snip creation:** After a rectangle is finalised, the client calls the stubbed `/api/snip-crop` endpoint with `{page, rect}`. The stub returns dummy OCR text and adds a new entry to the *SnipList*.
4. **Editing:** Users can edit the text in each box, insert new boxes, delete or merge boxes and undo/redo changes using keyboard shortcuts.
5. **Summarisation:** Clicking the **Summarize** button triggers a POST request to `/api/summarize` with the array of snippet texts; on success the summary is displayed.
6. **Export:** Users can copy the summary to the clipboard or export it as JSON or plain text. This action is local to the client and does not call the backend.

Implementation details

PDF upload and validation

`PDFUploader` uses HTML5 drag-and-drop and MUI's `Dropzone` to accept files. When a file is dropped/selected, its MIME type and size are checked. Unsupported file types or files larger than 10 MB trigger error messages, ensuring that only valid PDFs are processed.

```
function PDFUploader({ onLoad }) {
  const handleFiles = (files) => {
    const file = files[0];
    if (!file || file.type !== 'application/pdf') {
      setError('Unsupported file type. Please upload a PDF.');
      return;
    }
    if (file.size > 10 * 1024 * 1024) {
      setError('File exceeds 10 MB limit.');
      return;
    }
    setError(null);
    onLoad(file);
  };
  // Render drag-zone and fallback input
}
```

PDF viewer and snipping overlay

`PDFViewerPane` uses `<Worker>` and `<Viewer>` from *React PDF Viewer* to render the PDF. It attaches a custom `SnipOverlay` on top of each page canvas. The overlay listens for mouse down/move/up events to draw a semi-transparent rectangle with resize handles. Once drawing ends, it computes normalised coordinates:

```
function computeNormalizedRect(start, end, pageWidth, pageHeight) {
  const x = Math.min(start.x, end.x);
  const y = Math.min(start.y, end.y);
  const width = Math.abs(end.x - start.x);
  const height = Math.abs(end.y - start.y);
  return {
    x: x / pageWidth,
    y: y / pageHeight,
    width: width / pageWidth,
    height: height / pageHeight,
  };
}
```

After computing the rectangle, the overlay inverts the y-axis (`y_norm = 1 - (y + height)/pageHeight`) before sending the coordinates to the backend. Keyboard controls allow resizing and moving the rectangle; ARIA roles such as `role="button"` with `aria-label` are assigned to each handle.

Snip list and text editing

Each snip is an object `{id, page, rect, text}`. `SnipList` maps these objects to MUI `<TextField>` components and uses a `<Stack>` for vertical layout. Controls include:

- **Insert:** `Ctrl + N` or a plus icon inserts an empty text box.

- **Delete:** The **Delete** key or a trash icon removes the selected box.
- **Merge:** `Shift + M` merges the current box with the next one.
- **Undo/Redo:** `Ctrl + Z` and `Ctrl + Y` call `document.execCommand('undo')` / `redo`.

ARIA roles are added to the container (`role="list"`) and to each text box (`role="listitem"` with appropriate `tabindex`), enabling sequential keyboard navigation.

Summarisation workflow

The summarisation process collects text from all selected boxes and sends it to `/api/summarize`:

```
async function summarizeSnippets(snippets) {
  setLoading(true);
  try {
    const response = await fetch('/api/summarize', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ texts: snippets, max_sentences: 5 }),
    });
    const data = await response.json();
    setSummary(data.summary);
  } catch (err) {
    setError('Failed to summarize.');
  } finally {
    setLoading(false);
  }
}
```

The *Summarize* button triggers this function and shows a loading spinner. Pressing **Ctrl + Enter** triggers summarisation via keyboard. When a response is received, users can copy the summary via a copy icon or export it as a file using `Blob` and `URL.createObjectURL`.

Stubbed backend API

Because the backend OCR and summarisation services are under development, the front-end uses **Mock Service Worker (MSW)** to intercept API calls. The stubbed `/api/snip-crop` handler returns dummy OCR text and an image ID. The `/api/summarize` handler returns a canned summary. Additional stubs can simulate error cases or long-running operations; for example, returning a `500` status for invalid coordinates tests error handling. When the real API becomes available, remove MSW and update fetch URLs (e.g., `/pdf/snip-crop`, `/genai/summarize`).

Library evaluation (technical report)

The technical report compares several JavaScript libraries for rendering PDFs:

- **PDF.js:** A Mozilla project that renders PDFs using HTML5 and Canvas. It is free and widely used but offers only a basic UI; text selection can be inaccurate and the project sometimes lags behind new browser features.

- **React PDF:** A React wrapper around PDF.js providing `<Document>` and `<Page>` components but no full viewer UI. Developers must build thumbnails, toolbars and search features themselves.
- **React PDF Viewer:** A plugin-based React component built on PDF.js that offers drag-and-drop upload, thumbnails, zoom, search and full-screen view. It emphasises accessibility with WAI-ARIA roles and keyboard navigation. However, it depends on PDF.js internals and may need custom plugins for advanced annotation features.
- **Vue/Angular viewers:** Libraries such as **pdfvuer** and **ngx-extended-pdf-viewer** wrap PDF.js but target Vue or Angular applications; they are not relevant for a React application.
- **Commercial viewers:** Products like Apryse WebViewer integrate advanced annotations and forms. They provide professional support but introduce licensing costs and complexity not needed for the initial scope.

The report recommends **React PDF Viewer** because it delivers a rich UI out of the box, emphasises accessibility and exposes page scale information—critical for coordinate conversion during snipping.

Snipping and coordinate conversion

The technical report outlines best practices for implementing snipping:

- **Selection overlay:** Implement a transparent overlay on top of the PDF canvas. On mousedown start a drag, on mousemove update a bordered `div`, and on mouseup finalise coordinates.
- **Normalised coordinates:** Store selection coordinates as fractions of the displayed page size so the backend can multiply them by the actual page dimensions.
- **Coordinate inversion:** PDF coordinate systems originate at the bottom-left while browser canvases originate at the top-left. Invert the y-axis when converting: $y_p = H_p - (y_v + h_v) \times s_y$. Multiply by the DPI ratio when generating high-DPI images.
- **High-DPI cropping:** The backend can crop at 300 DPI using PyMuPDF's `page.get_pixmap(dpi=300, clip=rect)`. The front-end should send only normalised coordinates; precise cropping occurs on the server.

Editable text boxes and accessibility

The report emphasises that after OCR returns text, the UI should display it in editable boxes with rich editing controls. Keyboard navigation must follow W3C guidelines— `Tab` / `Shift + Tab` moves focus and arrow keys navigate within lists. Use appropriate ARIA roles (`role="list"` , `role="listitem"`) and ensure colour contrast meets WCAG 2.1 AA. Provide a high-contrast mode and ensure the drag-and-drop area is keyboard-operable.

Summarisation integration

After editing, the UI concatenates selected snippets and sends them to the `/genai/summarize` endpoint with an optional `max_sentences` parameter. Show a loading indicator while awaiting the response; display the summary below the text boxes or in a modal; and provide a keyboard shortcut (`Ctrl + Enter`) to trigger summarisation.

Asynchronous architecture and stubbed APIs

Although the front-end is the focus, understanding the server's asynchronous design informs UI design. FastAPI endpoints will be asynchronous and may delegate heavy tasks (OCR, summarisation) to background workers. For long-running operations, the server might return task IDs and allow clients to

poll or receive WebSocket notifications, but initial prototypes can simply wait for the HTTP response because OCR latency is expected to be ≤1.5 s. The front-end uses stubbed endpoints via MSW until the real services are available.

Security and compliance considerations

The UI contributes to overall security by:

- Validating MIME type and size before uploading; rejecting files with embedded scripts.
- Including CSRF tokens or using same-site cookies for state-changing operations.
- Providing meaningful error messages and allowing retry without losing user state.
- Ensuring accessibility compliance (WCAG 2.1 AA) with ARIA roles, keyboard navigation and proper focus management.

Testing and deployment

The technical report advocates automated testing and modern deployment practices:

- **OCR accuracy tests:** Maintain sample PDFs with expected OCR outputs to test cropping accuracy.
- **Docker multi-stage builds:** Build the React app in a Node image and copy the production build into a minimal web-server image (e.g. `nginx:alpine`) to reduce attack surface. Tag images with semantic versions and commit SHAs.
- **Continuous Integration/Continuous Deployment (CI/CD):** Use pipelines to build, test and deploy to staging and production. Use Kubernetes or similar orchestration to handle deployment and scaling.

Accessibility and internationalisation

Accessibility is a first-class concern. All interactive elements have ARIA roles, keyboard equivalents and visible focus outlines. Colour contrast meets a minimum ratio of 4.5:1 and users can toggle a high-contrast theme. Dynamic status messages use `aria-live` regions so screen readers announce updates.

Internationalisation support is achieved by storing UI strings in translation files (e.g. JSON) and passing them through a custom `useTranslation` hook. Adding language files enables future localisation.

Security considerations

Although the front-end cannot enforce server-side security, it contributes by:

- **Validating input:** Checking file type and size prevents malicious uploads.
- **Sanitising user content:** User-typed text is rendered as plain text rather than injected HTML, mitigating cross-site scripting (XSS).
- **CSRF protection:** Include CSRF tokens (via cookie or meta tag) in `X-CSRFToken` headers for state-changing requests.
- **HTTPS:** Assume secure connections; cookies use the `SameSite=Strict` attribute to prevent cross-site request forgery.
- **Error handling:** Catch network or API errors and display non-technical messages; allow users to retry without losing state.

Testing strategy

The front-end is designed for testability. **Unit tests** verify component behaviour, coordinate conversion and state updates. **End-to-end tests** (using tools like Cypress or Playwright) simulate uploading a PDF, drawing snips and summarising using the stubbed API. Accessibility tests run via `axe-core` to ensure WCAG compliance. Continuous integration pipelines build the React app, run tests and generate artefacts for deployment.

Setup and running the project

This project is a typical React application. To run it locally:

1. **Install dependencies:**

   ```
   npm install
   ```

2. **Start the development server:**

   ```
   npm start
   ```

   By default the app runs on `http://localhost:3000`. The development server sets `process.env.NODE_ENV` to `'development'`, enabling the Mock Service Worker.

3. **Build for production:**

   ```
   npm run build
   ```

   This generates a minified bundle in the `build/` directory. You can serve it with a static web server or copy it into a minimal Nginx container as described in the technical report.

4. **Configure API endpoints:** When the real backend becomes available, update the fetch URLs in `src/services/api.js` (or wherever API calls are defined) to point to the actual endpoints (e.g. `/pdf/snip-crop` and `/genai/summarize`) and remove MSW registration.

5. **Environment variables:** You may store the base API URL, maximum upload size and other configuration in a `.env` file. Use `process.env` in the code to access these variables.

Contributing

Contributions are welcome! Please ensure that your code adheres to the accessibility and security guidelines described above. When proposing changes:

1. Fork the repository and create a branch for your feature or bug fix.
2. Write unit tests and update documentation where applicable.
3. Submit a pull request. The CI pipeline will run tests and accessibility checks before merging.

License

Specify the license for the project here (e.g. MIT, Apache 2.0). Ensure that any third-party libraries are compatible with your chosen license.

Conclusion

This README provides a comprehensive overview of the **OCR Snipping & Summarization** front-end. By adopting React 18, React PDF Viewer and Material UI, implementing a modular architecture, emphasising accessibility and security, and following rigorous testing and deployment practices, developers can build a robust and user-friendly interface that meets the BRD’s requirements and integrates seamlessly with backend services.
